Ez **AngularJS modul létrehozása**:

```
angular.module('app1', [])
```

- **angular.module(...)**: az AngularJS modulkezelője.
- **'app1'**: a modul neve (alkalmazásunk „névtere”). Ezt fogjuk használni az oldalon:

  ```
  <html ng-app="app1">
  ```

- **[] (üres tömb)**: a modul **függőségeinek listája**. Üres = nincs másik modulra (pl. ngRoute) szükség a működéshez.
- Ha lennének függőségek:

  ```
  angular.module('app1', ['ngRoute', 'ngAnimate'])
  ```

Fontos különbség

- **Létrehozás/újradefiniálás**: angular.module('app1', [])
- **Lekérés (már létrehozott modul)**: angular.module('app1') ← itt **nincs** második paraméter!

Ha újra véletlenül []-t adunk, **felülírjuk** a modult (elveszhetnek a már regisztrált komponensek).

Gyors összefoglaló

- „Konténer” az alkalmazás részeihez (controller, service, directive, component, filter).
- A **második paraméter** (tömb) csak **egyszer**, a **létrehozáskor** kell.
- Az **ng-app="app1"** köti ezt a modult a HTML-hez, ezzel indul az app.

1) Profilkártya – élő előnézet

Mit tanít?

- **Kétirányú adatkötés**: ng-model ↔ interpoláció {{ ... }}.
- **Objektum-alapú űrlapmodell**: user objektum mezői (name, email, city, role, subscribed).
- **Származtatott nézet**: monogram generálása a névből.
- **Helyettesítő értékek** (üresen: „ismeretlen”, „—”, „nincs e-mail”).

Hol látható a kódban?

- ng-model="vm.user.name" stb. – minden űrlapelem a user-hez köt.
- {{ vm.user.name || 'ismeretlen' }} – feltételes kiírás.
- vm.initials(name) – névből monogram, nézetben hívva.

- Bootstrap: **form-control**, **form-select**, **card**, **card-header/body/footer**.

Mire figyeljünk?

- **Üres értékek kezelése**: mindig legyen fallback szöveg.
- **Függvény a sablonban**: kis, olcsó számítás mehet (monogram), de drága műveletek ne.
- **ControllerAs minta**: $scope helyett vm.

Tipikus hibák

- ng-model véletlenül sima változóra (pl. name) – jobb az objektum (pl. user.name) a könnyebb szerializáció és debug miatt.
- Funkció hívása túl gyakran (loopban), itt még rendben van.

---

2) ÁFA-s árkalkulátor (nettó → bruttó)

Miért jó ez a minta az AngularJS-es űrlapokhoz?

- ng-model néha **stringet** ad vissza, néha **numbert** (típus és valid állapot függvénye).
- A +vm.net || 0 **egységesen számmá** alakít és **véd** a hibás/üres bevitellel szemben.
- A Math.max(0, …) **üzleti szabályt** is érvényesít (ne legyen negatív).

Mit tanít?

- **Számított mezők** valós idejű frissítése.
- **Filter** használata formázásra: number.
- **Védett számítás**: negatív bevitelek kezelése (min. 0).

Hol látható?

- Függvények: netTotal(), discountAmount(), vatAmount(), grossTotal().
- Minden függvény a **nézetben hívva** jeleníti meg az értéket.
- Bootstrap rács, kártyák: vizuális tagolás az eredményekhez.

Mire figyeljünk?

- **Számok parse-olása**: +vm.net || 0.
- **Negatív értékek**: Math.max(0, ...).
- **Számítási sorrend**: kedvezmény a nettóra, ÁFA a kedvezmény utáni összegre.

Tipikus hibák

- ÁFA-t a kedvezmény előtti összegre számolják.
- Stringként kezelik a számokat, és „összefűződés” történik (pl. "10" + "2" → „102”).

---

3) Esemény-regisztráció – összegzés élőben

Mit tanít?

- **Vegyes inputtípusok**: text, date, select.
- **Selektált opció → objektum** (ng-options), nem csak string.
- **Szorzós árképzés** és kerekítés.

Hol látható?

- vm.tickets tömb: { name, factor, note }.
- ng-options="t as t.name for t in vm.tickets" – az egész objektum a modell.
- vm.finalPrice() – basePrice × ticket.factor, kerekítve.

Mit jelent az ng-options darabokra szedve?

- **for t in vm.tickets**
  Iterál a vm.tickets tömbön (Angular szinten), és minden elemre ad egy lokális változót: **t**.
- **t as t.name**
  - **t.name** lesz az **opció címkéje** a listában (amit a felhasználó lát).
  - **t** (az egész objektum) lesz az **opció értéke** (amit az ng-model megkap). Vagyis ha a felhasználó kiválaszt valamit, **vm.ticket maga a kiválasztott objektum** lesz, pl. { name:'Diák', factor:0.8, note:'-20%' }.

Mire figyeljünk?

- **Dátum**: üresen „nincs kiválasztva”.
- **Alapár védelem**: max(0, ...).
- **ng-options** előnye: a kiválasztott elem azonnal hordozza a szorzót és megjelenítendő nevet.

Tipikus hibák

- ng-model csak a névre (stringre) köt, aztán hiányzik a faktor.
- Ár kerekítése elmarad vagy rossz helyen történik.

---

4) Teendőlista – szűrés és számláló

Mit tanít?

- **Listakezelés és renderelés**: ng-repeat.
- **Szűrés** valós időben (saját filter callback függvény).
- **Állapotkezelés**: pipa done mezőre, vizuális állapot (áthúzás).
- **Tömeges művelet**: „kész elemek törlése”.

Hol látható?

- vm.items tömb: {text, done} elemek.
- ng-repeat="t in vm.items | filter:vm.search" – saját szűrőfunkció.
- ng-class – feltételes stílus (áthúzott).
- Bootstrap **list-group** + **btn**-ök a műveletekhez.

Mire figyeljünk?

- **Szűrő** legyen **kis-/nagybetű független** (lowercase összevetés).
- **track by** nem kötelező itt, de nagy listánál ajánlott.
- **Immutábilis törlés** nem feltétel, de tisztán tartja a gondolkodást (itt egyszerű splice / filter).

Tipikus hibák

- Szűrő az egész objektumon keres (zajos találatok) – szűrd item.text-re.
- Üres hozzáadás engedélyezett – kezeljük, hogy ne tegyen be üres elemet.

---

5) Termékkonfigurátor – élő ár és előnézet

Mit tanít?

- **Összetett árképzés**: több szorzó + fix felár egyszerre.
- **ng-style** dinamikus stílussal (szín-előnézet doboz).
- **Radio csoport (btn-group)** vs. select – különböző UI kontrollok összehangolása.

Hol látható?

- vm.colors tömb: { name, hex, factor } – selectből választva.

- Méret rádiók: S/M/L → sizeFactor() adja a szorzót.
- Extrák: két checkbox fix összegekkel.
- vm.finalPrice() – base × color × size + extras.

Mire figyeljünk?

- **ng-style**: {'background': vm.color.hex} – azonnali vizuális visszajelzés.
- **Százalékos kijelzés**: (factor*100 - 100)% – pozitív/negatív jelölés helyes.
- **Alapár védelem**: max(0, ...).

Tipikus hibák

- Szorzók egymás után rossz sorrendben? (Itt **szorzók kommutatívak**, sorrend lényegtelen.)
- vm.color üres, mielőtt a select beáll – kezdeti érték megadása fontos.